COS435 / ECE433: Introduction to RL

Due: May 8th, 10:30 PM

Taskmaster

Authors. Bryan Zhang (bz5989@princeton.edu), Itoro Ekpenyong (ie7780@princeton.edu), Jasamrit Rahala (jr3459@princeton.edu)

GitHub Repo (final): Github Repo

1 Introduction

How do we schedule tasks in an online setting, without being given true task lengths and only learned knowledge of how often tasks arrive? That is the fundamental problem we aim to tackle in this work, where we attempt to model general task assignments over time for an agent receiving multiple things to do over time. These tasks come at various rates, and the agent must learn to adapt to this cycle.

This problem can be best encapsulated into a simple example. Say that you are a student getting used to a new semester. You slowly get the hang of the rates that assignments come in, tests arrive, and readings are due. Suddenly, you need to schedule a certain task, i.e. an advisor meeting or a group project meetup. How can you figure out where to place this task? Intuitively, this problem splits into two parts. First, you must account for the amount of time this task will take. If the task takes longer, it would logically require a longer block of time in your schedule. Second, you must consider how long the tasks you currently have blocked off on the schedule might take. Certainly, if we presume knowledge of the true task lengths, we can freely schedule in the blank time spans, but if we only know an approximation of the true lengths, then we should leave leeway between events in case they take extra time. If you know that you have practice that may take somewhere from 2–3 hours and allocated a period of 2.5 hours for it, it is still better not to place something in those 30 minutes after practice and run the risk of needing to reschedule.

Translating to reinforcement learning, this becomes an agent that necessarily needs to learn the durations of tasks. This allows the agent to be able to best place a task, based on approximated true lengths of tasks. Intuitively, this is because while we may assume tasks take X amount of time, they often vary from this time. Thus, the agent can't rely on presumed time spans, but rather must learn an actual time span based on properties of the task. Second, we want the agent to learn a model of incoming task prediction over time, thus requiring effectively minimal changes to the schedules it produces. This is a very human want, as if every time we got a new task, we found that our prior planned schedule no longer works, there is no viability of planning for the future. Thus, we start with two criteria that follow from this problem: figuring out durations, then figuring out when best to place these new tasks as they come in into a schedule.

In essence, the agent, through these two properties of length prediction / modeling and keeping a close schedule, we have the agent learn to adapt to the future, given rates of tasks arriving. On this end, we presume the tasks do come at some reasonably predictable rates — while we don't assume the tasks of a certain type consistently come once every K time units, we do model them as coming at some consistent rate of arrival (i.e. every time unit, there is a 1/K chance of this task type being produced). This ensures there is a fundamental model for the agent to learn to estimate off of. Under this model of task arrival, the agent should learn to effectively schedule new tasks that arrive, based on what they see in the current history and the likelihood of getting conflicting events (with earlier due dates) at later time steps.

1.1 Prior Work

The foundational problem that this work is derived from is the job shop scheduling problem, where an agent must allocate a certain set of jobs to some set of machines, in order to target some goal [7]. These jobs have some order that should be satisfied, and tasks can't be partly completed. This fundamental problem is np-hard, as it requires an analysis of most plausible orderings to be assured that the assignments are reasonable.

One common prior target for the job shop scheduling problem is makespan — minimizing the total amount of time taken to do the set of jobs. Prior work in reinforcement learning has already aimed to tackle this type of problem, using RL as a means to counter the np-hard nature of the problem [6].

1.1.1 Rescheduling

When converted to the online problem, rather than preemptive scheduling, this becomes a problem of determining a next schedule at each time step [1]. Here, the critical issue is no longer to determine a best complete ordering, but rather how to handle modifications to the schedule as time goes by.

This brings the problem closer to the RL context, as the schedule must now be adaptive with respect to both the observed current environment and the remaining jobs. It also brings into consideration a different criteria — rather than just minimizing total time, we also want to minimize the amount of changes we are incurring. Prior work in RL has also analyzed this with respect to full schedules and accounting for plausible variations [5] from future job item changes.

A further change to the structure is by presuming less knowledge of the future — constant length tasks are presumed instead to come in at varying times and must be scheduled in RL Task Scheduler[2]. This introduces a form of environment with which an agent might interact — rather than just adversarial variations, there is a source of information.

1.1.2 Unknown Futures

These topics all help form the foundation to our work, but we take a step further and introduce another constraint on the agent, where we increase our lack of knowledge of the true tasks. This effectively acts as a different variation of the rescheduling structure, where rather than just variations on the already given tasks, as in the regular rescheduling problem, we additionally introduce jitter in the true lengths of tasks. This increases the difficulty of the problem by enforcing a greater need for an adaptive scheme: in the prior rescheduling problem, one could arguably get by — albeit inefficiently — with some variation of dynamic programming over the remaining tasks and in the future less structure, we could rely on partitioning the remaining space by the size of tasks, but here, the agent must truly build a model for scheduling and predicting tasks together.

As this increases the difficulty of the problem itself, we make certain simplifications towards the environment itself. We first presume a cyclic nature of task arrival: the arrival rates as noted early. This ensures there exists some pattern for the RL agents to learn from. We additionally only consider a single user that has to handle these tasks (rather than the multiple machines considered in the job-shop scheduling problem). To complete this simpler model, we presume that jobs arrive at sparser rates, as in the expected amount of new tasks gained each instant is less than 1 (to ensure that we don't overfill the schedules).

Combining these various properties, we have constructed a novel RL problem in the rescheduling space: concerning the scheduling and rescheduling of tasks over time as the agent adaptively learns of both new tasks at each time step (if any), as well as lengths of tasks through progress in them.

1.2 Overview

To tackle this problem, we built an environment for generating these systematic tasks as well as an agent that can model this problem and learn from it. While we initially wanted

to have human input, we decided for the ease of modeling to generate schedules instead, to ensure consistent or testable rates (without needing to have excessive manual input). For this structured environment, we allow for different types of tasks with varying properties, such as rates of arrival, penalties for lateness, and rough total allotted times. On the agent end, we use a PPO structure to train, basing off of Assignment 2, as well as incorporating a pointer network structure to analyze these complete prior schedules. We build our network off of the ideas noted in these two introductory structures that tackle the pointer network structure[4][3]. Major limitations as a result of this design are heavy variation in what form tasks take on due to our environment structure and an imbalance between PPO and the reassignment nature of the problem. As there is incredible variance in what a combined schedule and new task could take, we rely heavily on encoders to reduce this information to compressible, learnable forms, which comes at the cost of reduced transparency of what the agent ends up learning.

For results, we large focused on two analyses. We observe the effects of training on the produced schedules by analyzing where the agent chooses to place these tasks at different points in training. Given our trained model, we then also observe how our agent actually decides to schedule, observing where the final insertion policy places new tasks into the schedule when given more constraining task types.

2 Methods

We separately constructed an environment for generating tasks at a reasonable rate and an agent that observes tasks coming in at each time step and inserts into its own created schedule.

2.1 Environment

For the environment, we allow for different types of tasks, with each type carrying various properties, of which the most important are the mean time, the mean buffer time, and the mean reward. These determine, respectively, the expected amount of true time for the task, the expected amount of time the agent has before the assignment is due, and the expected reward drawn for the agent.

Additional to these task-specific parameters, we have the environment encode rates of occurrence for each task type — at every time step, the environment does a simple probability check to see if a task of each respective type is generated.

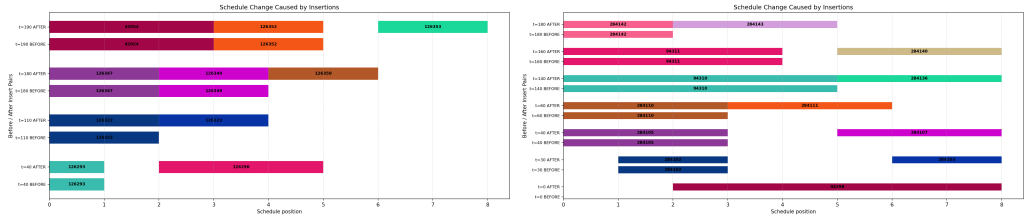
2.2 Agent

On the agent end, we segment into three parts: the internal model for shifting the schedule, the comprehensive pointer network actor-critic model that learns the schedule, and the general PPO learning structure.

For the internal schedule model, we end up internally modeling the full schedule, as that is the observable cumulative action that the agent takes. Compared to the general structure of an RL problem, where observations are of the full current state and actions are placed on this state, online scheduling entails a longer horizon of actions, where the agent effectively acts twice. Since the environment is only able to provide new tasks and mark lateness for old tasks, while the agent produces the true schedule, the agent needs to both do a task at a time step and schedule for the future. This makes for a more complicated learning process, as the rewards gotten at a time step often correspond to actions taken in much earlier time steps when tasks were initially inserted.

We specifically model the schedule as a horizon H space where the agent is, upon receiving a task, able to allocate unused space for said task. As the agent progresses through time steps, it shifts its schedule in that similar manner, acting upon what it has scheduled for the initial instance and receiving what might be a new task.

For the pointer network, we focus on the learning a few different parts: an encoder for the task, an encoder for the schedule, a value head, a duration head, and a layer



(a) Evaluation 20 (b) Evaluation 45

Figure 1: How the agent schedules tasks via insertion at different degrees of training

for corresponding a schedule with the tasks. This combination means that the agent is simultaneously learning properties of the tasks, schedules, and their interactions. The key combination for insertion is the agent using the encoded task and schedule with the pointer network layer in order to model where best to place a given task within a current schedule. In this insertion policy, the agent then masks invalid positions, before outputting some action as a form of placement of the new task, with penalties for failure to place. The agent additionally learns a value function for a schedule by chaining the schedule encoder with the value head, as well as a duration function by chaining the task encoder with the duration head.

We attempted to extend this pointer network structure to be a full replacement policy as well, but we were unable to figure out how to correctly model this replacement policy while still maintaining contiguous tasks. Namely, while we could use the pointer network to learn a full pointer change for every position, that resulted in our schedule incorrectly breaking up tasks into multiple parts. Varying by task chunks, however, broke the fixed action size constraint since the horizon was now variable size in number of blocks. This broke various properties, especially the action sets that PPO takes in, and as such, we proceeded with the rest of our work keeping the insertion policy instead of branching into a full replacement / reordering policy.

For our work, we generally kept the PPO structure intact, with the usual batch collection and PPO updates similar to what we wrote for Assignment 2. What changed was our focus on the inclusion of the duration loss as well, ensuring the model also learns to predict the task durations. This meant that overall, we used the same RL learning model as Assignment 2, with only internal plumbing changed to integrate better with the alternative environment and pointer network we used.

3 Results

3.1 Training

We largely focused our analysis on the effective outputs of our RL model in two forms, while training and a larger overview of how our agent acts when given more distinct tasks over time. In Figure 1, we can see the differences in how our model schedules tasks at two different time periods; Eval 20 and Eval 45. Both evidently have the agent learning the insertion policy, but they differ in how they schedule the tasks that they receive. In Eval 20, the agent is mostly scheduling at the immediate next instance, though occasionally leaving a bit of space. This is likely because of the penalties for lateness — it initially looks better to do things early to reduce the risk of lateness. In Eval 45, we see much more of the fruits of the learning. Here, the agent evidently is leaving more consistent gaps between tasks. Intuitively, this is a preemptive action in case a new, short task appears with an early deadline.

With regards to loss over time, in Figure 2, we observe that the model is able to rapidly learn the duration loss, quickly bringing that loss to near-zero values. For the value, the model is less capable of figuring it out, maintaining a large varied range. Intuitively, this makes sense, as it is difficult to evaluation the true value of the encoded schedule, as

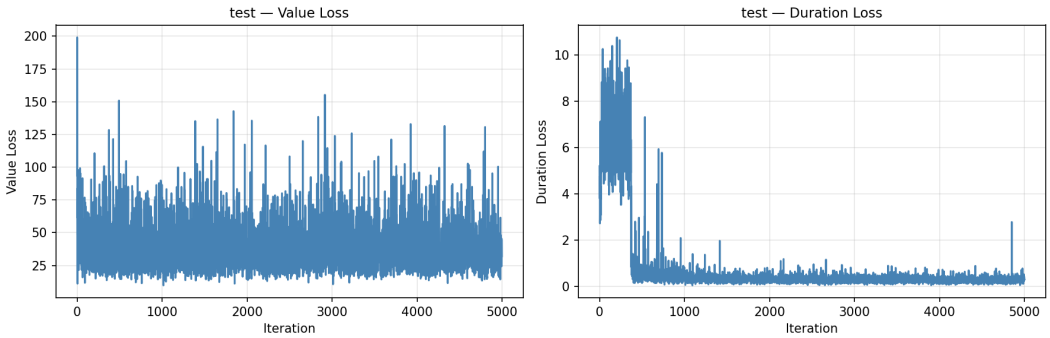


Figure 2: Value loss and duration loss while training

there is immense delay between action and schedule, as well as different effective value depending on what next task is given to be inputted.

3.2 Performance

For the trained model, we can more effectively analyze how our model actually performs over time. Here, we also make the tasks have a greater difference in size. In Figure 3, we can see how the agent schedules over a longer period of time, especially with regard to shorter and longer tasks. At t=11, for example, we observe that our agent leaves a significant gap between the beginning schedule position, choosing to place job 4 far away from jobs 0 and 2. At t=29, we see a similar thing occur where job 8 is placed much in the future of job 7. Here, it is evident that the agent is learning a degree of separation, plausibly in case a longer task arrives, such as long jobs 0 and 1. We can additionally observe at t=35 and t=43 that the scheduler doesn't just learn to space out the tasks, but also to fill in gaps where viable: the scheduler there proactively puts the tasks in earlier time slots that are free when it receives them, especially since the block becomes unlikely to be filled again after progressing a few more time steps in the future. This means that our insertion behavior is adapting to the different lengths and recognizing when it is or isn't possible to space out jobs to still prepare for the future.

4 Discussion and Limitations

4.1 Discussion

Overall our the graph analysis suggests that the learned scheduler learns to space out and prepare the existing schedule with gaps. While we do not have a full replacement policy, with critiques on schedule shifts, the agent does still seem to effectively learn spacing even with a simple negative reward for failure to fit in tasks into the current schedule.

We note, however, that this model does not generalize well to excessive task generation, likely following from the insertion rather than replacement policy, nor to generally different input from those upon which it trained. Namely, if we effectively increased the rate that tasks were generated, the model performed worse due to its space-opening nature, leaving many blocks unfilled.

We do observe in Figure 3, however, that the model does seem to focus on the task reward as well. There, short tasks were rewarded much more heavily than long tasks, at 20 vs 2 reward. We observed that the agent does aim to leave many open gaps, as if to allow short tasks that appear to be quickly filled in.

4.2 Limitations

Major limitations in our work lie in the way we make the predictions. We attempted to model this RL problem as best as we could on a real task assignment problem, where we

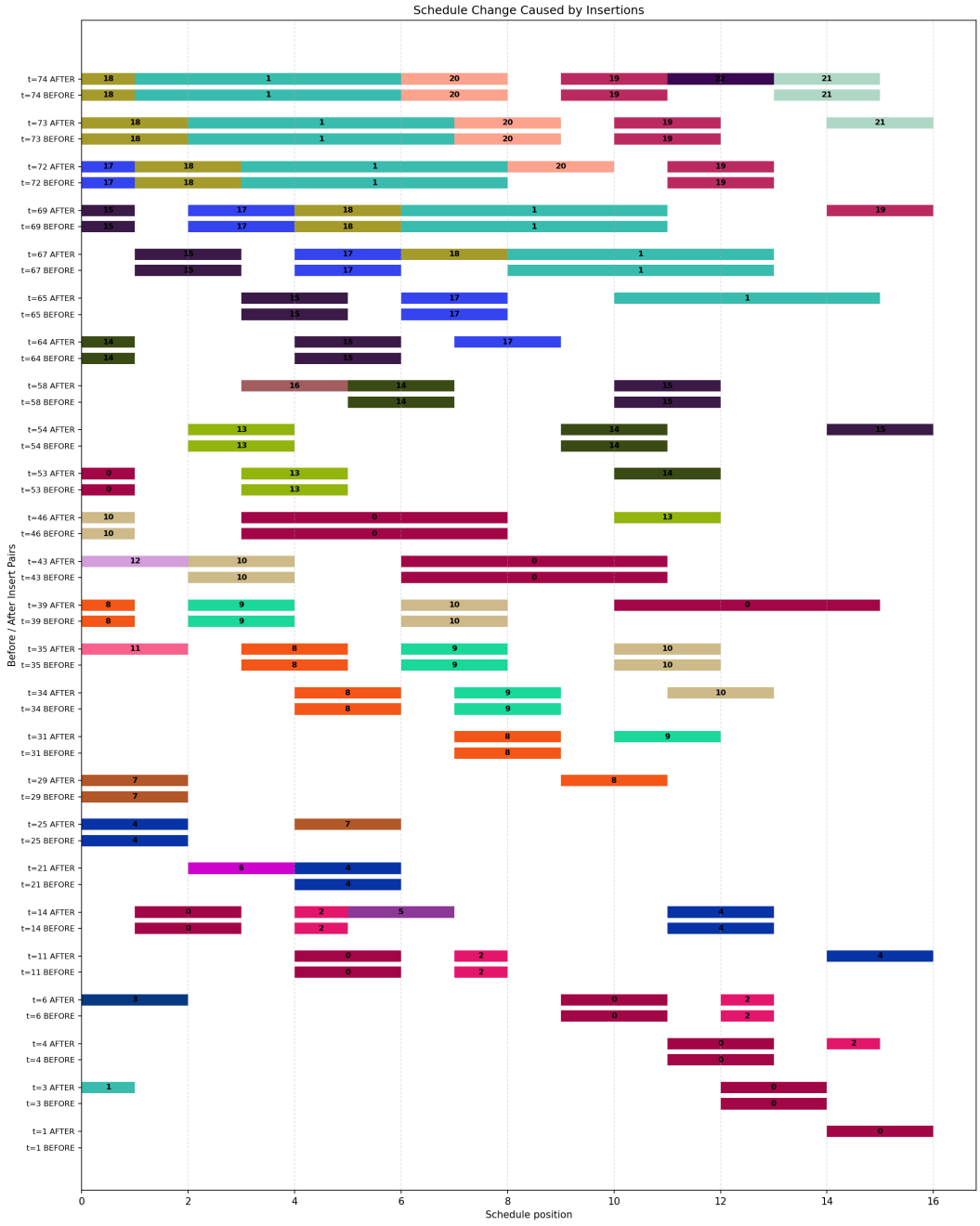


Figure 3: Varying length tasks over time. Here, short tasks had reward 20 and long tasks only had reward 2

get tasks at a reasonable rate (i.e. homework) and need to figure out time points to place these sorts of tasks. However, this requires numerous parameters for realism, many of which we put aside in order to be able to effectively consider the problem with the training structure. Many plausibly reasonable inclusions, such as continuously increasing costs for working longer and longer, fall apart in the batched PPO structure. Additionally, it was difficult to ensure simultaneous training happened for many of the relevant parameters — for duration, we ultimately took the direction of following the actor-critic model of appending different heads for determining the policy and value, and included a further

head for duration. Thus, there isn't true simultaneous learning — only effectively bridging off the learning into subsidiary network layers.

Another limitation is that the agent never really learns how to care about overloading. Due to its preemptive spacing and since we were unable to flesh out a replacement procedure, the agent is unable to handle later increased density of tasks with many small gaps in between.

4.2.1 Future Work

Future extensions that would be useful would be the replacement of the current insertion action policy with a true insert and replace policy, allowing the agent to modify the schedule while adding in the new task. This allows for an additional penalty for schedule shift, relaxing the current constraint of maintaining the schedule with a weaker constraint of costly rescheduling, which models the real world better.

References

[1] Mohammad Mehdi Ahmadian, Amir Salehipour, and T. C. Edwin Cheng. A meta-heuristic to solve the just-in-time job-shop scheduling problem. *European Journal of Operational Research*, 288(1):14–29, 2021.

[2] Artemisak. rl-task-scheduler. `https://github.com/artemisak/rl-task-scheduler`, 2026. GitHub repository, accessed May 8, 2026.

[3] FastML. Introduction to pointer networks. `https://fastml.com/introduction-to-pointer-networks/`, 2017. Accessed May 8, 2026.

[4] Higgsfield. Intro to pointer network. `https://github.com/higgsfield/np-hard-deep-reinforcement-learning/blob/master/Intro%20to%20Pointer%20Network.ipynb`, 2026. GitHub notebook, accessed May 8, 2026.

[5] Sumin Hwangbo, J. Jay Liu, Jun-Hyung Ryu, Ho Jae Lee, and Jonggeol Na. Production rescheduling via explorative reinforcement learning while considering nervousness. *Computers & Chemical Engineering*, 186:108700, 2024.

[6] Prosysscience. Rl-job-shop-scheduling. `https://github.com/prosysscience/RL-Job-Shop-Scheduling`, 2026. GitHub repository, accessed May 8, 2026.

[7] Pankaj Sharma and Ajai Jain. A review on job shop scheduling with setup times. *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture*, 230(3):517–533, 2016.